

プロジェクトデータ構造・処理フロー・ロジック解説

このプロジェクトは、静的な統計データ（人流）と動的なルーティング処理、そしてハードコードされた観光地データを組み合わせて、京都の観光案内と混雑状況の予測を提供しています。

1. データ元とデータ構造

このプロジェクトでは、主に以下の3種類のデータを扱っています。

A. 人流統計データ (human_flow_monthly.json)

京都エリアの「いつ、どこに人が多いか」を記録した巨大な統計データです。

- ソース: オープンデータ（2019-2021年の3年間の月別平均データと推測されます）。
- 構造階層:
 - 月 (Month): 1月～12月
 - 曜日区分 (DayType): 平日 (weekday) / 休日 (weekend)
 - 地域メッシュ (Mesh Code): 緯度経度を約1km四方に区切ったID (例: 52347768)
 - 時間帯 (TimeOfDay): 朝 (morning) / 昼 (noon) / 夕方 (evening)
 - 値: その条件下での推計人口数
- 目的: 観光地ごとの「混雑レベル (1～5)」を動的に算出するために使用されます。

B. 観光地データ (services/spotService.ts)

アプリ内に直接記述（ハードコード）された観光スポットのデータベースです。

- 内容: 名前、説明、緯度経度、デフォルトの混雑度、営業時間など。
- 特徴: RAW_SPOTS 配列として約70～80件のデータが定義されています。

C. 交通・ルーティングデータ (public/data/ 内のJSONファイル)

公共交通機関の経路探索に使用されるデータ群です。

- kyoto-stops-graph.json : バスの停留所と接続情報のグラフデータ。
- kyoto-bus-routes.json : バスの運行ルートの形状データ（地図上に線を描画するため）。
- timetables/{routeId}.json : バスの時刻表データ（リアルな出発時刻計算用）。

2. データ処理とスクリプト (Logic) の解説

データ処理の中心となる「スクリプト」は、主に services ディレクトリ内のTypeScriptファイルに実装されています。

① 人流・混雑度計算ロジック (humanFlowService.ts)

このスクリプトは、「緯度経度」を「混雑レベル(1～5)」に変換するエンジンです。

- メッシュコード変換 (latLonToMesh1km)
 - 処理: JIS X 0410規格に基づき、緯度・経度から「地域メッシュコード (8桁)」を計算します。
 - 解説: 緯度を1.5倍、経度から100を引くなどの計算を行い、1次メッシュ→2次メッシュ→3次メッシュ(約1km四方)へと絞り込みます。
- 統計データの取得 (getMeshPopulation)
 - 処理: 計算したメッシュコードと、現在時刻 (月・曜日・時間帯) をキーにして、JSONから人口数を取得します。
- 偏差値によるレベル判定 (populationToCongestionLevel)

- **処理:** 単純な人数ではなく、観光スポット全体の人口分布から**偏差値 (Deviation Score)** を算出し、相対的な混雑度を決定しています。
 - **レベル1 (快適):** 偏差値 42未満 (下位20%)
 - **レベル3 (通常):** 偏差値 48-52 (中間層)
 - **レベル5 (混雑):** 偏差値 58以上 (上位20%)
- **工夫点:** 事前に登録された観光スポットの場所だけの統計を取り (`registerSpotLocations`), 関係ない住宅街などのデータが含まれないように正規化しています。

② ルーティング・経路探索ロジック (`routeService.ts`)

このスクリプトは、「現在地」から「目的地」への最適バス・徒歩ルートを提案する複雑なエンジンです。

1. 候補ルートの選定 (`searchRoutes Phase 1`)

- **処理:** 出発地と目的地の近くにあるバス停を検索し、それらを結ぶ「共通の系統 (Route ID)」をリストアップします。
- **スコアリング:** 直線距離ベースで概算所要時間を計算し、有望なルート上位8件に絞り込みます。

2. 詳細経路と時刻計算 (`searchRoutes Phase 2`)

- **徒歩ルート取得 (`getWalkingPath`):** バス停までの徒歩ルートを `OSRM (Open Source Routing Machine)` APIから取得します。失敗した場合は直線距離でフォールバックします。
- **時刻表照合 (`findNextBus`):**
 - `timetables/` データを読み込み、徒歩移動時間を考慮して「次に乗れるバス」を検索します。
 - もし今日のバスが終わってれば、**翌日の始発**を検索するロジック(`isTomorrow` フラグ)も実装されています。
 - 時刻表がない路線 (または地下鉄) の場合、時間帯に応じたランダムな待ち時間をシミュレーションするロジックが含まれています。

3. ルート形状の切り出し (`getBusPath`)

- **処理:** `kyoto-bus-routes.json` にある長いバスルートの形状データから、「乗車バス停」から「降車バス停」までの区間だけを切り出します。
- **工夫点:** バス停の位置関係から進行方向を判定し、逆方向の場合は座標配列を反転(`reverse`)させる処理が入っています。

3. 処理後のデータ (Processed Data) について

これらの処理を経て、アプリ上でユーザーに提供されるデータは以下のようになります。

1. 動的混雑レベル:

- 単なる「混んでいる/空いている」ではなく、「平日・昼間のこの場所は、他の観光地と比較してレベル3」といった、文脈 (コンテキスト) を考慮した精度の高い指標になります。

2. リアルな移動プラン (`RouteOption`):

- 「徒歩X分 → バスY分 → 徒歩Z分」という構成だけでなく、「XX:XX発のバス」という具体的な時刻付きの案内になります。
- 地図上には、バス停までの徒歩ルートと、バスが実際に通る道路の形状が正確に描画されます。